上位机工作链与通信调用图

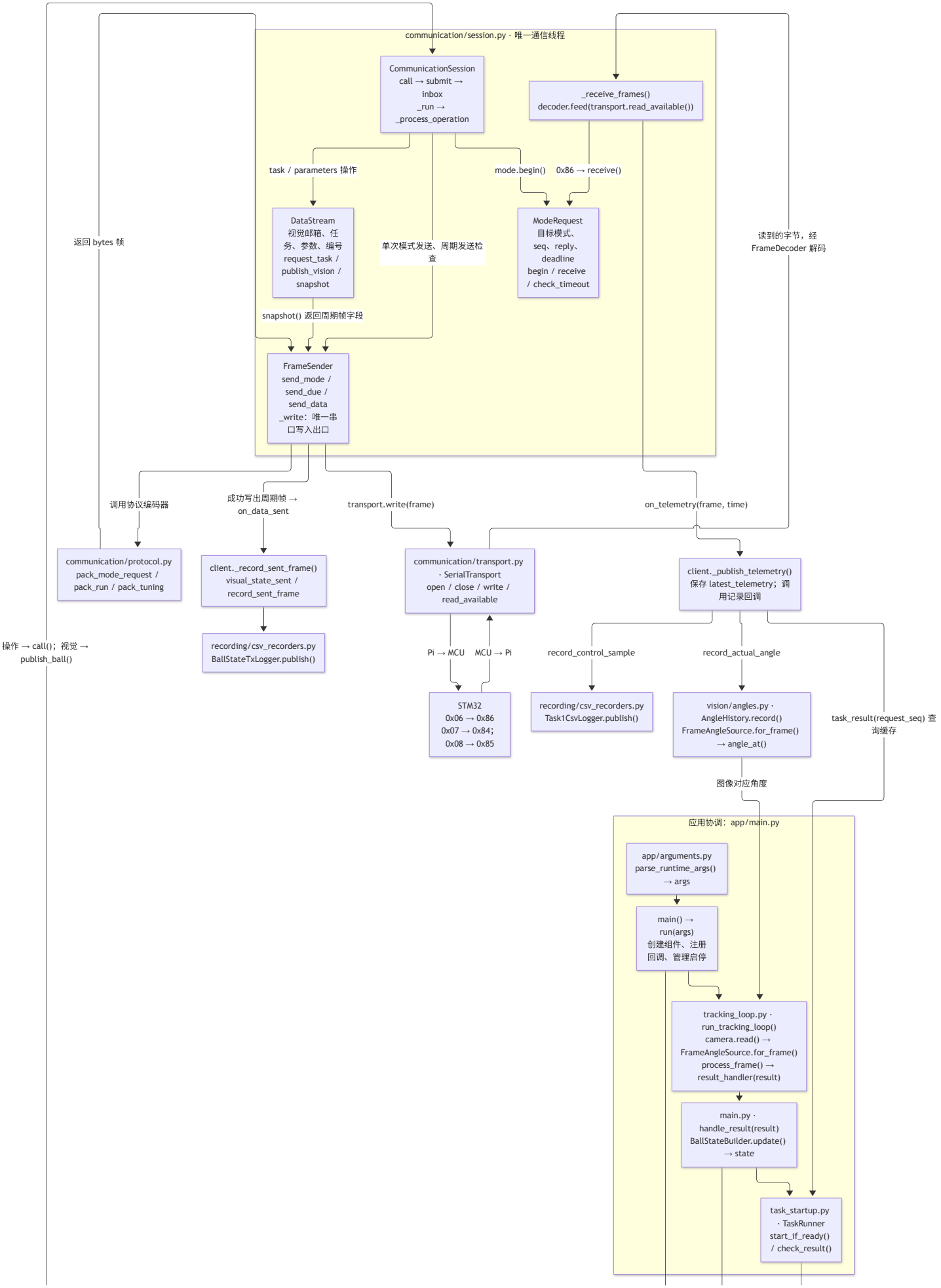
核对日期：2026-10-20。对应当前源码；本文不修改运行行为。

先区分目录与运行职责

`app/main.py` 是应用组装与协调入口，不是所有组件所在的目录。命令行定义在 `app/arguments.py`，视觉算法在 `vision/`，记录器在 `recording/`，串口设备访问目前仍在 `communication/transport.py`，尚未迁移到 `hardware/`。

本文图中箭头上的文字是具体调用或数据交接。带“队列”的边是线程交接，不是串口发送。`reply` 是 Python Future，不是 MCU 应答帧。

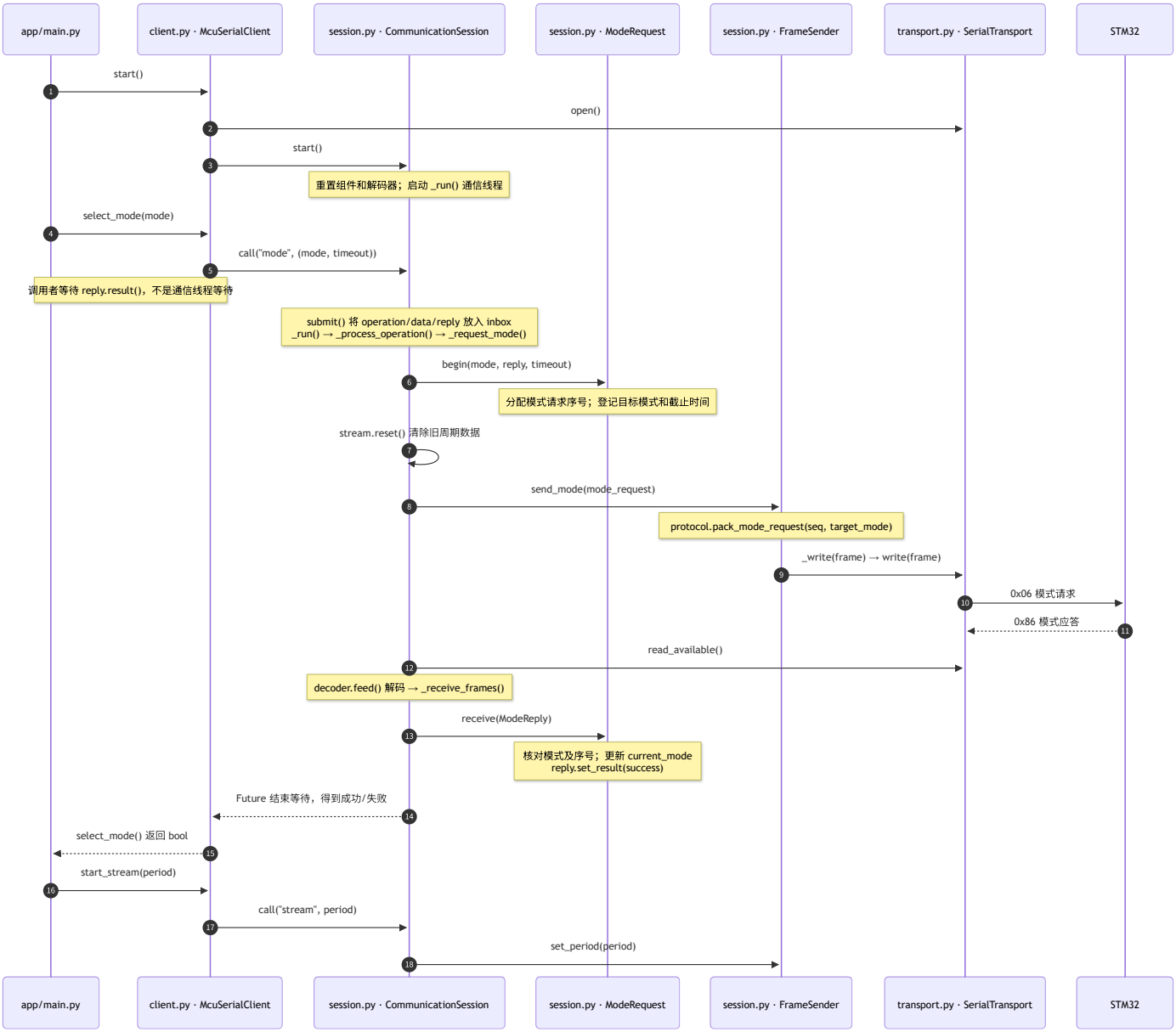
一、整体闭环


communication/session.py · 唯一通信线程
CommunicationSession
call → submit → inbox
_run → _process_operation
_receive_frames()
decoder.feed(transport.read_available())
task / parameters 操作
mode.begin()
0x86 → receive()
DataStream
视觉邮箱、任务、参数、编号
request_task / publish_vision / snapshot
ModeRequest
目标模式、seq、reply、deadline
begin / receive / check_timeout
返回 bytes 帧
单次模式发送、周期发送检查
读到的字节，经 FrameDecoder 解码
snapshot() 返回周期帧字段
FrameSender
send_mode / send_due / send_data
_write：唯一串口写入出口
调用协议编码器
成功写出周期帧 → on_data_sent
transport.write(frame)
on_telemetry(frame, time)
communication/protocol.py
pack_mode_request / pack_run / pack_tuning
client._record_sent_frame()
visual_state_sent / record_sent_frame
communication/transport.py · SerialTransport
open / close / write / read_available
client._publish_telemetry()
保存 latest_telemetry；调用记录回调
操作 → call()；视觉 → publish_ball()
recording/csv_recorders.py
BallStateTxLogger.publish()
Pi → MCU
MCU → Pi
record_control_sample
record_actual_angle
STM32
0x06 → 0x86
0x07 → 0x84；
0x08 → 0x85
recording/csv_recorders.py
Task1CsvLogger.publish()
vision/angles.py · AngleHistory.record()
FrameAngleSource.for_frame() → angle_at()
task_result(request_seq) 查询缓存
图像对应角度
应用协调：app/main.py
app/arguments.py
parse_runtime_args() → args
main() → run(args)
创建组件、注册回调、管理启停
tracking_loop.py · run_tracking_loop()
camera.read() → FrameAngleSource.for_frame()
process_frame() → result_handler(result)
main.py · handle_result(result)
BallStateBuilder.update() → state
task_startup.py · TaskRunner
start_if_ready() / check_result()



`FrameDecoder` 在 `protocol.py` 中工作，接收流程由 session 调用。模式应答不交给 `_publish_telemetry()`；模式管理器填写 Future，使等待中的 `select_mode()` 返回。

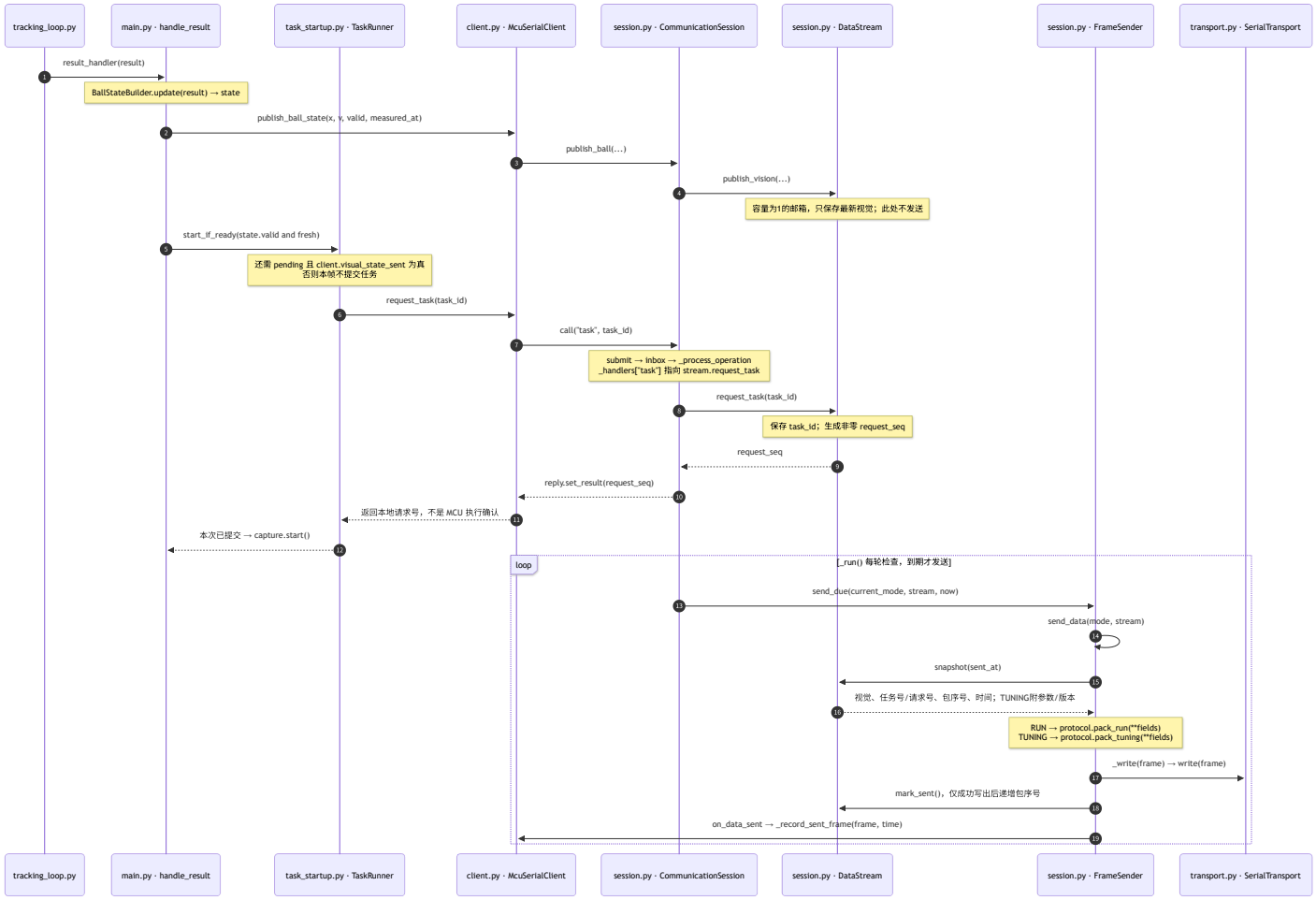
二、启动、模式请求与确认



- `begin()` 只登记，不阻塞；模式请求只发一次，不自动重试。
- 周期帧只有在模式已确认且 period 非 None 时才发送。

- 如果写串口失败， `_request_mode()` 调用 `mode.fail()` ；若一直没有匹配应答，通信循环调用 `check_timeout()` 返回失败。

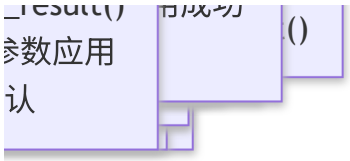
三、视觉更新、任务登记与周期组帧



周期发送可以早于任务请求： `request_seq=0` 表示没有任务动作，仍可传送视觉。请求登记后，后续周期包重复携带相同任务请求号；MCU 按请求号去重。不是每次周期发送都创建新任务请求。

`set_velocity_parameters()` 走同样的操作队列： `call("parameters", VelocityParameters(...))` → `DataStream.set_parameters()` 。它只保存参数并增加 revision，由后续 TUNING 周期包携带，不立即单独发一帧。

四、遥测返回、结果查询与角度反馈



查询结果只读客户端缓存，不发送“查询串口包”。这里的任务成功是 MCU 接受并执行启动操作，不代表整个运动任务已经结束。角度历史使用 Pi 接收时刻，不能直接当作 MCU 实际采样时刻。

五、三个记录入口不能混为一条

flowchart TB OPEN["recording/capture.py · CaptureSession.open()
创建可选记录器及共享 CsvWriteWorker"] RESULT["main.handle_result(result)
capture.record_vision(result, state)"] --> PROBE["csv_recorders.py
VisionProbeLogger.publish()
视觉结果及提交给通信的小球状态"] SENT["FrameSender 成功写出周期帧
→ client._record_sent_frame()
→ record_sent_frame(packet, sent_at)"] --> TX["csv_recorders.py
BallStateTxLogger.publish()
实际成功写出的周期包"] RX["client._publish_telemetry()
→ record_control_sample(sample)"] --> CTRL["csv_recorders.py
Task1CsvLogger.publish()
MCU 回传控制采样"] OPEN -.-> PROBE OPEN -.-> TX OPEN -.-> CTRL PROBE --> WRITE["recording/csv_writer.py
CsvWriter.write() → CsvWriteWorker.submit()
后台线程写 CSV"] TX --> WRITE CTRL --> WRITE

`main.run()` 注册回调：`client.record_actual_angle = angles.record`、`client.record_control_sample = capture.control.publish`、`client.record_sent_frame = capture.sent.publish`。这些赋值不执行记录函数，等收到遥测或成功写出周期帧后才调用。

六、按源码回顾的顺序

1. main.py：`main → run`，看组件创建、回调赋值和 `handle_result()`。
2. arguments.py：`parse_runtime_args()` 生成 args；config.py 合并配置与命令行。
3. task_startup.py：`select_task()` 选择 TaskSpec；`initialize_task_startup()` 做 task 0 和专用初始化；`TaskRunner` 管启动请求状态。
4. client.py：上层入口、创建通信组件、保存遥测、交付记录回调。
5. session.py：先读 `_run()`，再读 `_process_operation()` 和 `_request_mode()`；随后读三个内部组件。

6. protocol.py：字段布局、编码解码和 FrameDecoder；transport.py：字节读写。
7. tracking_setup.py： `build_tracking_components()` 创建 camera、calibration、tracker；内部调用 detector_setup.load_detector()。
8. tracking_loop.py：取图、匹配角度、调用 tracker、交出 result；ball_state.py：提取提交给通信的小球状态。
9. angles.py：收到的角度如何缓存，再按图像时刻查询、插值。
10. capture.py：记录协调；csv_recorders.py：各 CSV 的列；csv_writer.py：后台写盘。

七、线程边界与容易混淆的状态

内容	执行位置或含义
args 解析、setup、视觉循环、TaskRunner	app 调用侧；不会因调用 client 自动变成通信线程
submit/call	调用者把操作交给通信线程；call 等待 Future
_run、模式确认、任务字段更新、串口读写	唯一 McuCommunication 通信线程
publish_ball → publish_vision	视觉调用侧更新线程安全邮箱，不等 MCU、不直接发包
_publish_telemetry、_record_sent_frame 及记录回调	由通信线程调用；配套记录器快速入队，不在此等待磁盘
CSV 写盘	共享 CsvWriteWorker 后台线程
visual_state_sent	最近一次成功写出的周期包视觉有效位；不是 MCU 接收确认
request_seq	任务请求编号；同一请求在多个周期帧内重复携带
packet_seq	成功写出的周期包序号；每次成功写出递增
revision	参数版本；用于区分不同参数更新的应用结果
reply	本地 Future；模式操作等模式应答，task 操作只等本地登记

普通退出由 ExitStack 逆序停止相关组件、停止周期发送和通信、关闭文件。当前 task-console 的 Ctrl+C 使用 os._exit(130) 立即退出，不保证执行这些清理或排空 CSV；需手动复位下位机。